

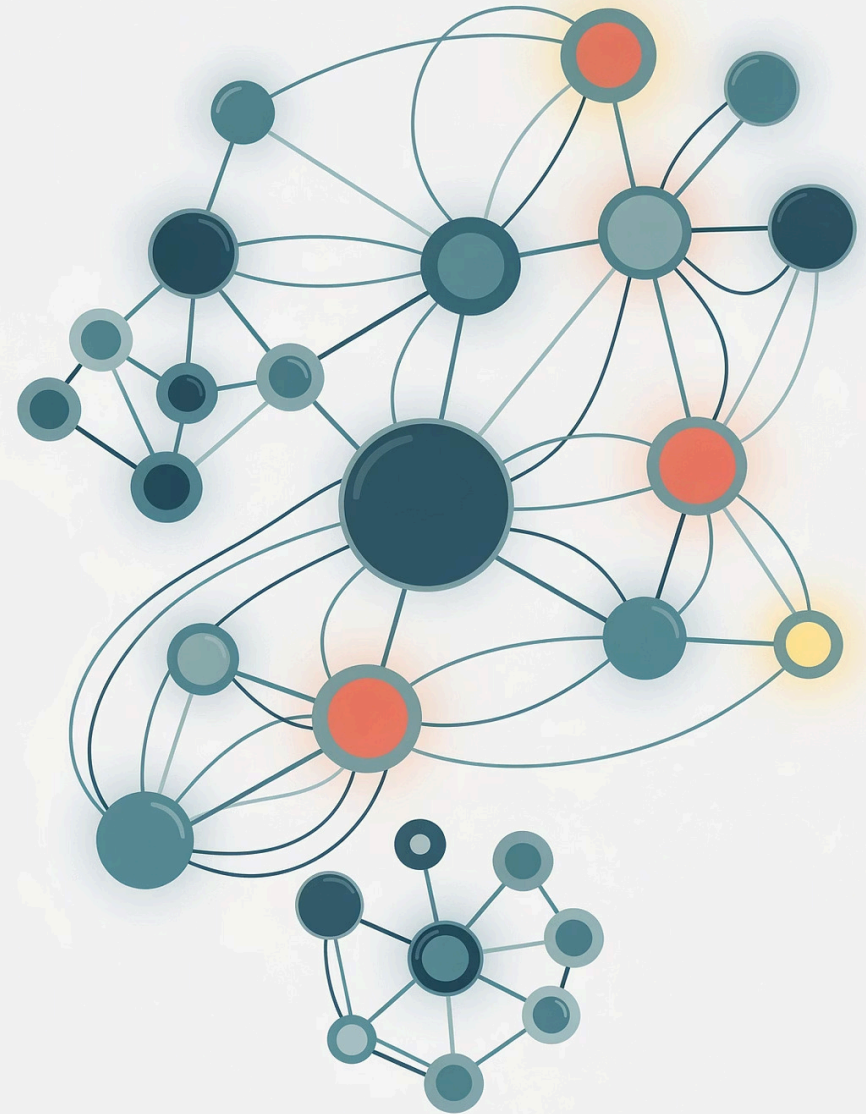
Graph Theory — Advanced Module

A deep dive into **Weighted Graphs** and **Minimum Spanning Trees** — the foundational structures behind real-world network design, routing algorithms, and optimisation problems in computer science.

ADVANCED DSA

GRAPH ALGORITHMS

COMPUTER SCIENCE



PART 1: Weighted Graphs

Section 1 — Topic Classification

Module Overview

Topic: Weighted Graphs
Category: Graph Theory / Data Structures
Domain: Computer Science / Graph Algorithms
Difficulty: Advanced DSA

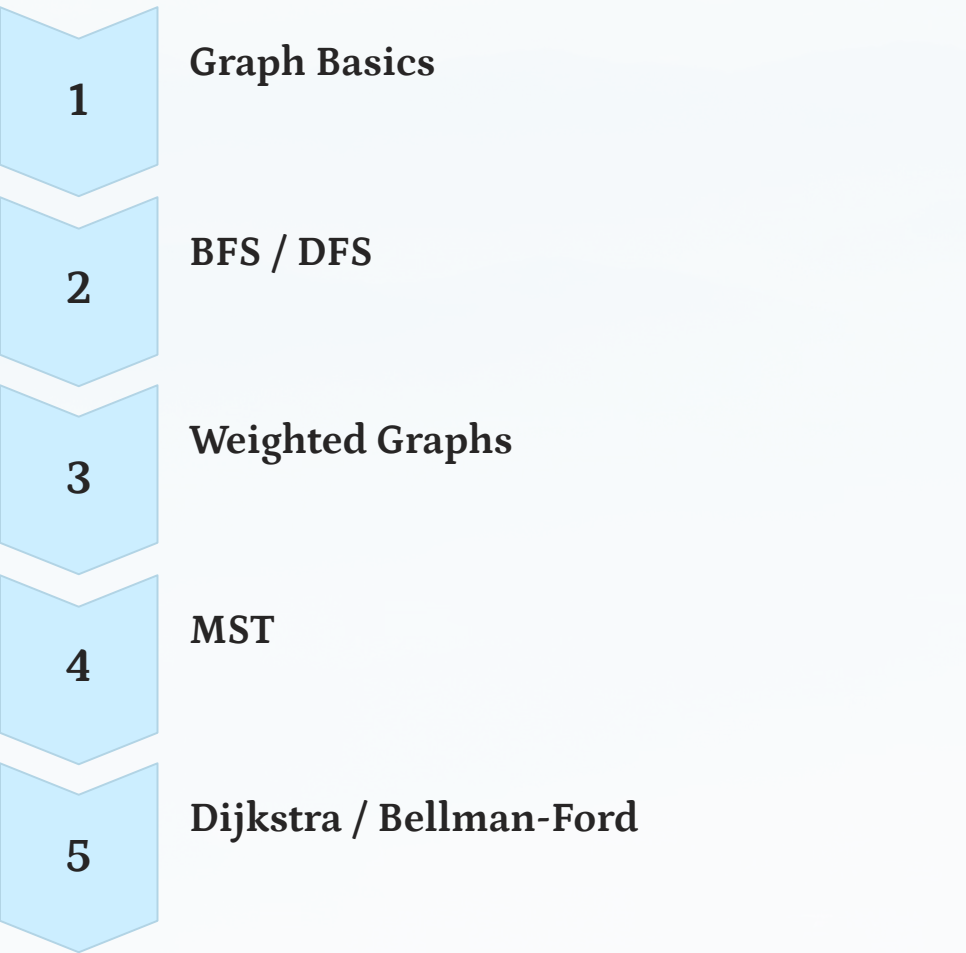
Prerequisites

- Graph Basics (adjacency list/matrix)
- BFS and DFS traversal
- Directed and Undirected Graphs

Industry Relevance

Critical — Every real-world graph problem involves weights: road maps (distance/time), networks (bandwidth/latency), logistics (cost/capacity), social graphs (similarity/influence).

Recommended Learning Path



What Is a Weighted Graph?

Section 2 — Formal Definition

A **weighted graph** $G = (V, E, w)$ extends the standard graph definition with a **weight function** $w: E \rightarrow \mathbb{R}$ that assigns a real-valued weight to every edge. For an edge (u, v) , its weight is written as $w(u, v)$ or $c(u, v)$ (cost).

Weights can represent **distance, time, cost, capacity, probability, similarity**, or any quantifiable relationship. The semantics depend entirely on the problem domain.

i For weighted undirected graphs: $w(u,v) = w(v,u)$ (symmetric). For directed graphs: $w(u,v) \neq w(v,u)$ in general. Convention: $w(u,v) = \infty$ if edge $(u,v) \notin E$.

Path weight: For path $P = v_0, v_1, \dots, v_k$:
 $w(P) = \sum w(v_i, v_{i+1})$ for $i = 0$ to $k-1$

Edge Weight Semantics by Domain

Domain	Vertices	Weight Meaning
Road Network	Intersections	Distance (km) or time (min)
Computer Network	Routers/Servers	Bandwidth (Mbps) or latency (ms)
Electrical Grid	Power stations	Resistance (Ω) or capacity (MW)
Social Network	Users	Friendship strength / interaction count
Logistics	Warehouses	Shipping cost (\$) or delivery time
NLP Word Graph	Words	Pointwise Mutual Information (PMI)

Representations of Weighted Graphs

Section 3 — Three Core Representations

Adjacency List

SPARSE GRAPHS

Each entry stores (neighbour, weight) pairs.

adj[u] = list of (v, w) pairs

- Space: $O(V + E)$
- Add edge: $O(1)$
- Iterate neighbours of u: $O(\text{degree}(u))$
- Check if (u,v) exists: $O(\text{degree}(u))$

✔ Best for: BFS, DFS, Dijkstra

Adjacency Matrix

DENSE GRAPHS

matrix[u][v] = weight; ∞ if no edge; 0 on diagonal.

- Space: $O(V^2)$
- Add edge: $O(1)$
- Check (u,v) + get weight: $O(1)$ ← key advantage
- Iterate neighbours of u: $O(V)$

✔ Best for: Floyd-Warshall, $O(1)$ lookup

Edge List

MST ALGORITHMS

Flat list of (u, v, weight) triples.

- Space: $O(E)$
- Sort by weight: $O(E \log E)$ ← critical for Kruskal's
- Check if (u,v) exists: $O(E)$
- Get all edges: $O(E)$ — efficient iteration

✔ Best for: Kruskal's MST algorithm

Representation Decision Framework

Scenario	Best Representation	Reason
Sparse graph ($E \approx V$)	Adjacency List	Saves memory
Dense graph ($E \approx V^2$)	Adjacency Matrix	$O(1)$ weight lookup
MST algorithms	Edge List	Sort by weight
Dijkstra / BFS	Adj. List + Min-Heap	Efficient neighbour iteration
Floyd-Warshall (all-pairs)	Adjacency Matrix	Access all pairs efficiently

Important Weighted Graph Concepts

Section 4 — Paths, Negative Weights, and Algorithm Selection

Path Weight and Shortest/Longest Paths

Given a weighted graph $G = (V, E, w)$:

- **Shortest path** from s to t : $P^* = \operatorname{argmin} w(P)$ over all s - t paths
- **Longest path** on general graphs: NP-Hard; on DAGs: $O(V+E)$ via DP
- **SSSP** (Single-Source Shortest Path): Dijkstra $O((V+E) \log V)$
- **APSP** (All-Pairs Shortest Path): Floyd-Warshall $O(V^3)$

Edge Weight Classification → Algorithm

Weight Type	Algorithm	Complexity
All weights = 1	BFS	$O(V+E)$
Weights 0 or 1	0-1 BFS (deque)	$O(V+E)$
Non-negative reals	Dijkstra	$O((V+E) \log V)$
Arbitrary reals	Bellman-Ford	$O(VE)$
All-pairs	Floyd-Warshall	$O(V^3)$

Negative Weights — A Crucial Distinction

 Dijkstra's algorithm **FAILS** with negative weights — it can return incorrect distances. Always check for negative weights before choosing your algorithm.

Non-negative weights only ($w(e) \geq 0$): Dijkstra is correct and efficient.

Negative weights (some $w(e) < 0$): Use Bellman-Ford — $O(VE)$.

 **Negative Cycle:** A path where total weight < 0 that loops back on itself. Shortest path is **undefined** — you can loop forever, decreasing weight.

Example: $A \rightarrow (3) \rightarrow B \rightarrow (-5) \rightarrow C \rightarrow (1) \rightarrow A$
Cycle weight = $3 + (-5) + 1 = -1 < 0$
Detected by: Bellman-Ford in $O(VE)$

Key Problems on Weighted Graphs

Section 5 — Five Canonical Problems

- | | |
|---|--|
| 1 | Minimum Spanning Tree
Scenario: Connect n cities with roads at minimum total construction cost. This is the central topic of this module.
Algorithms: Prim's, Kruskal's |
| 2 | Shortest Path
Scenario: GPS navigation — find the fastest route from A to B.
Algorithms: Dijkstra (no negative weights), Bellman-Ford (any weights), A* (with heuristic) |
| 3 | Maximum Flow / Minimum Cut
Scenario: Water flowing through pipes with capacities. Maximise flow from source to sink. Weights represent capacities .
Algorithms: Ford-Fulkerson, Edmonds-Karp |
| 4 | Travelling Salesman Problem (TSP)
Scenario: Visit all cities exactly once and return to start at minimum total cost. NP-Hard on general weighted graphs.
Approximation: Christofides algorithm — 1.5× optimal |
| 5 | Negative Cycle Detection
Scenario: Currency arbitrage — detect if any sequence of exchanges yields profit. Model: $\log(\text{exchange_rate})$ as negative edge weight. Negative cycle = arbitrage opportunity.
Algorithm: Bellman-Ford in $O(VE)$ |

Implementation — Weighted Graph Class

Section 6 — Pseudocode Reference

```
CLASS WeightedGraph:
  FUNCTION __init__(V, directed=FALSE):
    self.V = V
    self.directed = directed
    self.adj = [[] for _ in range(V)]
    self.edges = [] // (u, v, w) triples

  FUNCTION add_edge(u, v, w):
    self.adj[u].append((v, w))
    self.edges.append((u, v, w))
    IF NOT self.directed:
      self.adj[v].append((u, w))
      self.edges.append((v, u, w))

  FUNCTION neighbors(u):
    RETURN self.adj[u] // List of (v, w) pairs

  FUNCTION total_weight():
    IF NOT self.directed:
      RETURN sum(w for u,v,w in self.edges) / 2
    RETURN sum(w for u,v,w in self.edges)
```

```
FUNCTION sorted_edges():
  // Returns edges sorted by weight
  // Used by Kruskal's algorithm
  unique = self.edges if self.directed
    else self.edges[:len(self.edges)//2]
  RETURN sorted(unique, key=lambda e: e[2])

FUNCTION to_matrix():
  INF = float('inf')
  mat = [[INF]*V for _ in range(V)]
  FOR u in range(V):
    mat[u][u] = 0
  FOR (u,v,w) in self.edges:
    mat[u][v] = w
  RETURN mat
```

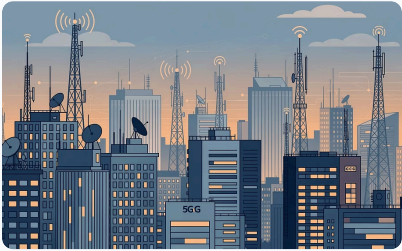


Key design decisions:

- Adjacency list stores (neighbour, weight) pairs
- Edge list stores (u, v, w) triples for Kruskal's
- Undirected edges added in both directions
- `sorted_edges()` deduplicates for undirected graphs

Real-World Use Cases

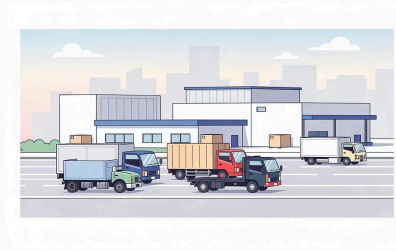
Section 7 — Industry Applications of Weighted Graphs



Telecommunications Network Design

Context: ISP building a fibre network across 50 cities.

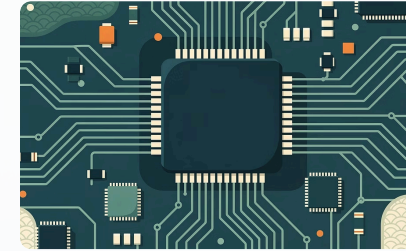
- Cities = vertices; fibre routes = weighted edges (w = installation cost in \$M)
- **Problem:** Connect all cities with minimum total cable cost
- **Solution:** MST on the weighted graph
- **Constraint twist:** Pre-laid conduits $\rightarrow$ constrained MST or Steiner tree variant



Logistics and Supply Chain

Context: Amazon distribution network.

- Warehouses + fulfilment centres = vertices; shipping lanes = directed weighted edges
- MST variant: minimum infrastructure cost to keep all nodes connected
- SSSP: fastest route for a specific shipment
- Max-flow: maximum throughput on a given route



VLSI Circuit Design

Context: Intel/AMD chip layout.

- Logic gates = vertices; wires = weighted edges (w = wire length $\rightarrow$ signal delay + power)
- **Problem:** Connect all gates with minimum total wire length
- **Solution:** MST, then refined with Steiner tree algorithms (allow extra junction points)

Hands-On Lab — Build a Weighted Graph

Section 8 — Compute Properties from Scratch

Graph Specification

0-indexed, undirected graph with 6 vertices.

Edges:

(0,1,4), (0,2,3), (1,2,1), (1,3,2),
(2,4,6), (3,4,5), (3,5,7), (4,5,8)

Expected Observations

- Total weight = $4+3+1+2+6+5+7+8 = \mathbf{36}$
- Min edge: (1,2,**1**)
- Max edge: (4,5,**8**)
- Weighted degree of vertex 1: $4+1+2 = \mathbf{7}$
- DFS from 0 (one possible order): 0,1,2,4,5,3

Lab Tasks

01

Build Representations

Construct both adjacency list and adjacency matrix representations of the graph.

02

Compute Degree Sequence

Find the sorted degree sequence. Also compute the **weighted degree** (sum of adjacent edge weights) for each vertex.

03

Identify Min/Max Edges

Find the edge with minimum weight and the edge with maximum weight.

04

Run DFS from Vertex 0

Record the order of discovery and the total weight of tree edges used during traversal.

05

Verify Matrix Structure

Confirm that in-order traversal on the adjacency matrix gives the same graph structure.

Common Mistakes — Weighted Graphs

Section 16 — Pitfalls and Fixes

Using BFS for Weighted Shortest Path

Impact: Wrong answer — BFS counts hops, not weight.

Fix: Use Dijkstra (non-negative weights) or Bellman-Ford (any weights).

Applying Dijkstra with Negative Weights

Impact: Incorrect distances returned.

Fix: Check for negative weights first; use Bellman-Ford if any exist.

Adjacency Matrix with 0 for "No Edge"

Impact: Cannot distinguish "no edge" from a "zero-weight edge".

Fix: Use ∞ (or None/-1) for missing edges; reserve 0 only for self-loops.

Storing Undirected Edges Once but Treating as Directed

Impact: Traversal misses half the edges.

Fix: For undirected graphs, always add both (u,v,w) and (v,u,w) .

Double-Counting Edges in Total Weight

Impact: Sum is $2\times$ the actual total weight.

Fix: Divide by 2, or store each undirected edge exactly once in the edge list.

Learning Summary — Weighted Graphs

Section 18 — Key Takeaways and Quick Reference

1 Weighted Graph Definition

$G = (V, E, w)$ adds weight function $w: E \rightarrow \mathbb{R}$ to the standard graph definition.

2 Representation Choice

Adjacency List (sparse, BFS/DFS/Dijkstra) · Matrix (dense, Floyd-Warshall, $O(1)$ lookup) · Edge List (Kruskal's, sorting by weight).

3 Negative Weights Invalidate Dijkstra

Use Bellman-Ford $O(VE)$ for any weights. Negative cycles make shortest path undefined — always check first.

4 Five Canonical Problems

MST · SSSP · APSP · Max-Flow · TSP — each with distinct algorithms and complexity bounds.

Weighted Graphs Quick Reference

$G = (V, E, w)$ — $w: E \rightarrow \mathbb{R}$ assigns weight to each edge

Adj. List: $O(V+E)$ space | $O(\deg)$ neighbour iter ✓ BFS

Adj. Matrix: $O(V^2)$ space | $O(1)$ weight lookup ✓ Floyd

Edge List: $O(E)$ space | $O(E \log E)$ sort ✓ MST

Path weight = sum of edge weights on the path

All weights=1: BFS $O(V+E)$ | **Non-negative:** Dijkstra $O((V+E) \log V)$

Any weights: Bellman-Ford $O(VE)$ | **All-pairs:** Floyd-Warshall $O(V^3)$



PART 2: Minimum Spanning Trees

Section 1 — Topic Classification

Module Overview

Topic: Minimum Spanning Tree (MST)

Category: Graph Algorithms

Domain: Computer Science / Greedy Algorithms / Graph Theory

Difficulty: Advanced DSA

Industry Relevance

Critical — Network design, circuit layout, cluster analysis, approximation algorithms for NP-hard problems.

Prerequisites

Weighted Graphs

Union-Find (DSU)

Priority Queues (Min-Heap)

Greedy Algorithm Paradigm

Recommended Learning Path

1

Weighted Graphs

2

MST Theory

3

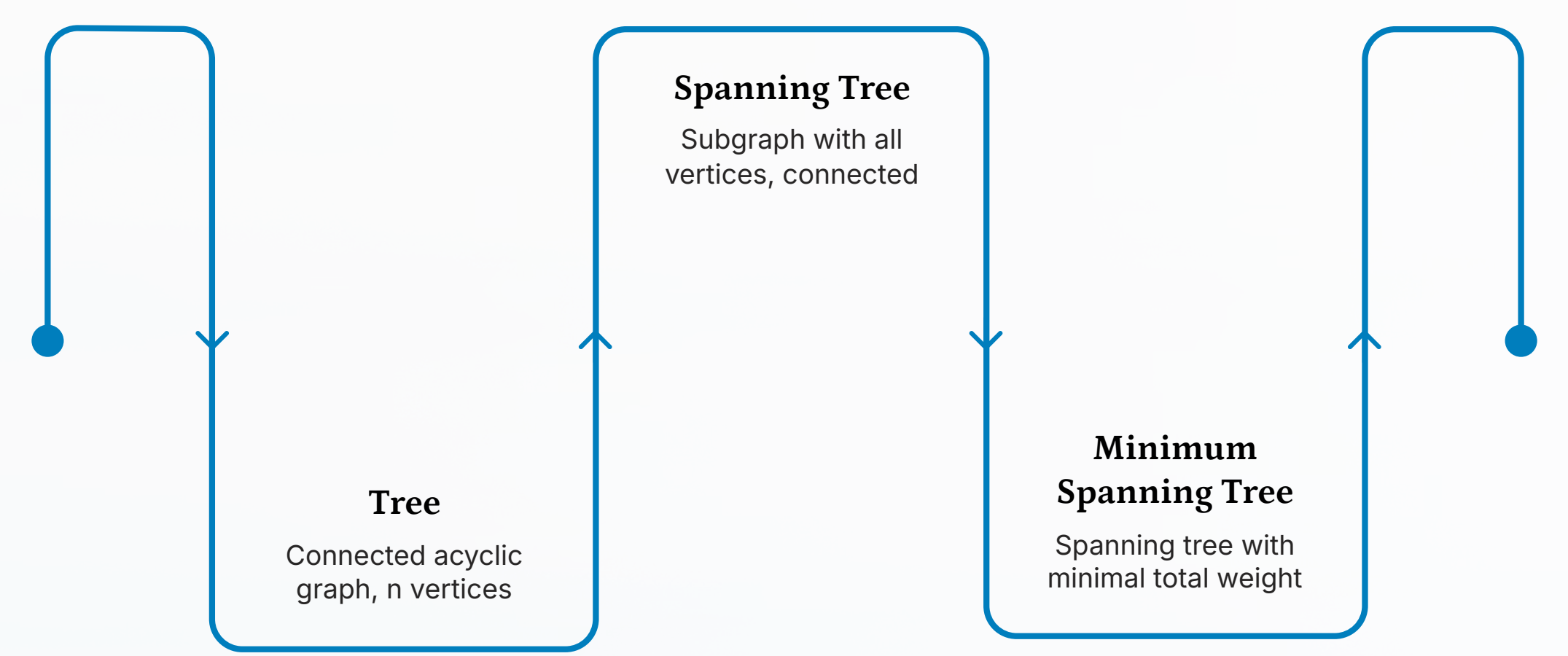
Prim's Algorithm

4

Kruskal's Algorithm

What Is a Spanning Tree?

Section 2 — Definition Chain: Tree → Spanning Tree → MST



The MST is the spanning tree whose total edge weight is minimised over all possible spanning trees of G .

Key Properties of MSTs

Property 1 — Uniqueness

If all edge weights are distinct, the MST is unique. With duplicate weights, multiple MSTs may exist but all have the **same total weight**.

Property 2 — $n-1$ Edges

Any spanning tree of a connected graph with n vertices has **exactly $n-1$ edges**.

Property 3 — Cycle Property

For any cycle C in G , the **maximum weight edge** of C does NOT belong to any MST.

Property 4 — Cut Property

For any cut $(S, V-S)$ of G , the **minimum weight edge** crossing the cut belongs to some MST. This is the foundation of greedy MST algorithms.

Why the Cut Property Works

i Both Prim's and Kruskal's greedily select minimum-weight edges that don't form cycles. The Cut Property **guarantees** these greedy choices are always safe — they will always be part of some valid MST.

Cycle Property — Proof Intuition

If the max-weight edge e^* of cycle C were in MST T , removing e^* disconnects T into two components. But C provides another path between those components (going around the cycle without e^*). Replacing e^* with any other edge in C gives a spanning tree of smaller or equal weight — contradicting that T is the MST. **QED**

Theoretical Foundations

Section 3 — Generic MST Algorithm and Matroid Theory

The Generic MST Algorithm

Both Prim's and Kruskal's are instances of a single generic greedy framework:

```
GENERIC-MST(G, w):
  A = ∅ // A will become the MST edge set
  WHILE A does not form a spanning tree:
    Find an edge (u,v) that is SAFE to add to A
    A = A ∪ {(u,v)}
  RETURN A

// "Safe" edge: adding it keeps A ⊆ some MST
// The Cut Property tells us: min-weight edge
// crossing any cut that respects A is always safe.
```

MST Complexity Landscape

Algorithm	Complexity
Borůvka	$O(E \log V)$
Prim's (binary heap)	$O((V+E) \log V)$
Kruskal's	$O(E \log E) = O(E \log V)$
Prim's (Fibonacci heap)	$O(E + V \log V)$
Karger-Klein-Tarjan	$O(E)$ randomised — optimal!
Chazelle	$O(E \alpha(V))$ — nearly linear
Lower bound	$\Omega(E)$ — must examine every edge

Why Greedy Works — Matroid Theory

❑ The reason greedy algorithms give optimal MSTs is that the set of spanning forests of a graph forms a **graphic matroid**. Greedy on a weighted matroid always finds the optimal solution.

Matroid $M = (E, I)$ where E = set of all edges and I = set of all acyclic edge subsets (forests).

Hereditary

Any subset of an acyclic set is also acyclic.

Exchange Property

If $|A| < |B|$ and both are acyclic, some edge of B can be added to A keeping it acyclic.

MST is exactly the **minimum weight basis** of the graphic matroid. This is why both Prim's and Kruskal's are provably correct — they are both greedy algorithms on a matroid.

Cut Property — Deep Dive

Section 4 — The Theoretical Backbone of MST Algorithms

 Understanding the Cut Property deeply separates good DSA engineers from great ones. It is the single theorem that justifies the correctness of every greedy MST algorithm.

Definitions

Cut $(S, V \setminus S)$

A partition of V into two non-empty sets S and $V \setminus S$.

Crossing Edge

An edge with one endpoint in S and the other in $V \setminus S$.

Respects A

A cut respects a set A of edges if no edge in A crosses the cut.

Light Edge

The minimum weight edge crossing a given cut.

Cut Property Theorem: Let $A \subseteq$ MST edges found so far. Let $(S, V \setminus S)$ be a cut that respects A . Let $e =$ light edge crossing $(S, V \setminus S)$. Then e is **SAFE** to add to A .

Proof Sketch

01

Start with any MST T

If $e \in T$, we are done immediately.

02

If $e \notin T$

T has a path p from u to v (endpoints of e). Since $u \in S$ and $v \in V \setminus S$, path p must cross the cut somewhere. Let e' be that crossing edge.

03

Compare Weights

$w(e) \leq w(e')$ because e is the light edge. Form $T' = T - \{e'\} + \{e\}$, which is also a spanning tree.

04

Conclude

$w(T') = w(T) - w(e') + w(e) \leq w(T)$. Since T is MST, $w(T') \geq w(T)$, so $w(T') = w(T)$. Therefore T' is also an MST containing e . 

 **Cycle Property (equally fundamental):** For any cycle C in G , the maximum weight edge of C does not belong to any MST. Kruskal's implicitly applies this — the skipped edge would be the heaviest in some cycle.

MST Variants and Extensions

Section 5 — Beyond the Standard MST

Maximum Spanning Tree

Find the spanning tree with **maximum** total edge weight.

Solution: Negate all weights → run MST → negate back. Or sort edges in *decreasing* order and run Kruskal's.

Application: Maximum reliability network.

Minimum Spanning Forest

When the graph is **disconnected**, no spanning tree exists. Find one MST per connected component.

Both Prim's and Kruskal's handle this naturally. Kruskal's run to completion produces the minimum spanning forest.

k-Minimum Spanning Tree

Find the minimum weight subtree spanning exactly **k vertices** (not necessarily all).

Complexity: NP-Hard in general. $O(\log k)$ approximation via iterated greedy. Exact: DP on tree subsets — exponential.

Steiner Tree

Connect a required subset of **terminal vertices** at minimum cost. Additional non-terminal Steiner points may be used.

Application: VLSI routing — connect k pins using junction points to minimise wire length. NP-Hard in general; polynomial on graphs: $O(3^k \times V + 2^k \times V^2)$.

MST Under Edge Constraints

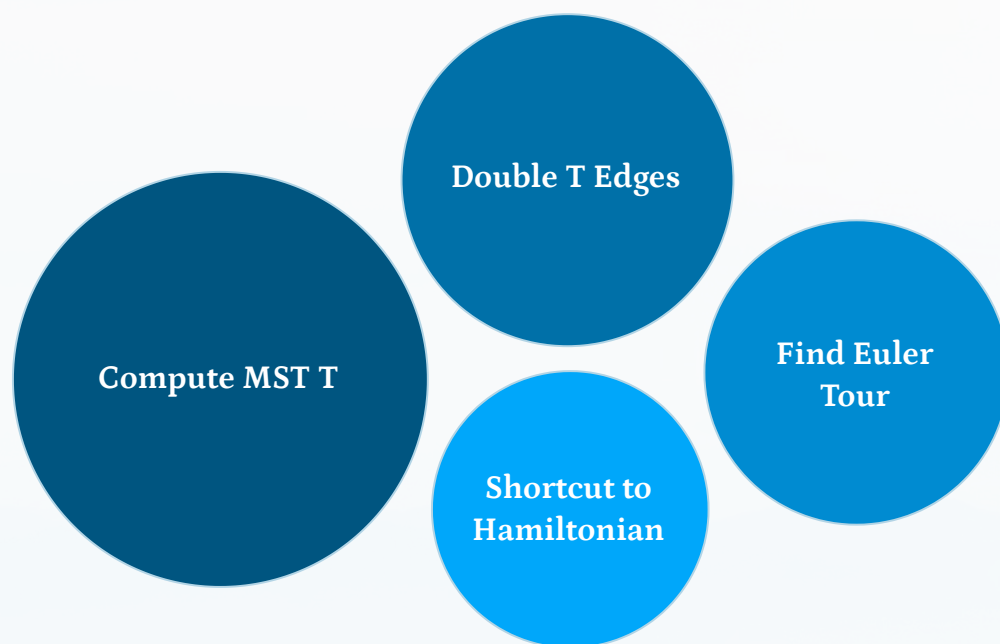
- **Degree-Constrained MST:** No vertex degree $> d$. NP-Hard.
- **Minimum Bottleneck ST:** Minimise the maximum edge weight. Any MST is also an MBST!
- **Capacitated MST:** Edges have capacity constraints. Relevant for bandwidth-limited network design.

MST in Approximation Algorithms

Section 6 — MST as the Backbone of Hard Problem Solvers

2-Approximation for Metric TSP

Theorem: On a metric graph (edge weights satisfy triangle inequality), the following algorithm gives a tour $\leq 2 \times \text{OPT}$.



Result: Hamiltonian cycle of weight $\leq 2 \times w(\text{MST}) \leq 2 \times \text{OPT}$.

Christofides Algorithm (1.5-Approximation)

01

Compute MST T

Standard MST on the metric graph.

02

Find Odd-Degree Vertices O

$|O|$ is always even by the handshaking lemma.

03

Minimum Weight Perfect Matching M on O

$w(M) \leq (1/2) \times \text{OPT}$ by triangle inequality.

04

Combine T + M $\rightarrow$ Euler Tour $\rightarrow$ Shortcut

Every vertex now has even degree. Find Euler tour, then shortcut repeated vertices.

✓ Weight $\leq w(\text{MST}) + w(M) \leq \text{OPT} + (1/2)\text{OPT} = \mathbf{1.5 \times \text{OPT}}$.
This was the best known ratio for 47 years. In 2021, Karlin-Klein-Oveis Gharan achieved $1.5 - \epsilon$.

Single-Linkage Clustering via MST

Build MST of fully connected weighted graph ($w(u,v)$ = distance). To get k clusters: remove the $k-1$ most expensive MST edges. Remaining connected components = clusters. Equivalent to agglomerative hierarchical clustering, computable in $O(n^2 \log n)$.

Hands-On Labs — MST Theory

Section 7 — Prove the Cut Property and Sensitivity Analysis

Lab 1: Prove the Cut Property on a Concrete Graph

Graph: Vertices $\{0,1,2,3,4\}$

Edges: $(0,1,8)$, $(0,2,5)$, $(1,2,9)$, $(1,3,11)$, $(2,3,15)$, $(2,4,10)$, $(3,4,7)$

Tasks

1. Find all spanning trees with weight < 50
2. Identify the MST manually by trying all combinations
3. For cut $S=\{0,2\}$, $V \setminus S=\{1,3,4\}$: list all crossing edges; identify the light edge; verify it's in your MST
4. For cut $S=\{0,1,2\}$, $V \setminus S=\{3,4\}$: repeat the above
5. Find cycle 2-3-4-2. Which edge does the Cycle Property say is NOT in the MST? Verify.

Lab 2: MST Sensitivity Analysis

Same graph as Lab 1.

01

Find the MST

Record its total weight as the baseline.

02

Increase Heaviest MST Edge by 1

Does the MST change? Why or why not?

03

Decrease a Non-MST Edge

At what weight does it enter the MST? This is the **bottleneck weight** — the max weight on the unique path between its endpoints in the MST.

04

Implement `mst_sensitivity(graph, edge)`

Returns the range [decrease, increase] of the given edge's weight for which the MST remains unchanged.

Common Mistakes — Minimum Spanning Trees

Section 16 — Pitfalls and Fixes

Confusing Spanning Tree with Shortest Path Tree

Impact: Completely different structures used interchangeably.

Fix: MST minimises *total edge weight*; SPT minimises *path weights from a source*. They are fundamentally different.

Assuming MST Gives Shortest Paths

Impact: Wrong distances used in routing or navigation.

Fix: $\text{MST} \neq \text{SSSP tree}$. Use Dijkstra's algorithm for shortest paths.

Applying MST to Directed Graphs Naively

Impact: MST is defined for undirected graphs only.

Fix: For directed graphs, use **Minimum Spanning Arborescence** (Edmonds' / Chu-Liu algorithm).

Forgetting to Check Graph Connectivity

Impact: Algorithm produces a forest, not a spanning tree.

Fix: Check that the result has exactly $V-1$ edges. If fewer, the graph was disconnected.

Ignoring Equal-Weight Edges

Impact: Non-unique MST; algorithm-specific behaviour may cause confusion.

Fix: Equal weights $\rightarrow$ multiple valid MSTs exist, but all have the **same total weight**. Any is acceptable.

Learning Summary — Minimum Spanning Trees

Section 18 — Key Takeaways, Quick Reference, and Interview Questions

1 Spanning Tree Definition

Connected acyclic subgraph containing ALL vertices; always has exactly $n-1$ edges.

2 MST Definition

Spanning tree with minimum total edge weight. May not be unique if weights repeat, but all MSTs share the same total weight.

3 Cut Property

The min-weight edge crossing any cut that respects current MST edges is safe to add. This justifies both Prim's and Kruskal's greedy choices.

4 Cycle Property

The max-weight edge of any cycle is NOT in any MST. This justifies Kruskal's cycle-skipping behaviour.

5 Applications

Network design · 2× TSP approximation · Christofides 1.5× TSP · Single-linkage clustering · VLSI layout · Steiner tree approximation.

MST Quick Reference

Prim's (binary heap): $O((V+E) \log V)$ | **Kruskal's:** $O(E \log E) = O(E \log V)$

Prim's (Fibonacci heap): $O(E + V \log V)$ ← best for dense graphs | **Lower bound:** $\Omega(E)$

Common Interview Questions

Cut Property

What is it? How does it prove greedy MST algorithms are correct?

Cycle Property

What is it? How does Kruskal's implicitly use it?

MBST Proof

Prove that any MST of G is also a Minimum Bottleneck Spanning Tree.

Equal Weights

If all edge weights are equal, how many spanning trees does K_n have? (Cayley's formula: $n^{(n-2)}$)

Maximum ST

How do you find a Maximum Spanning Tree?

MST and TSP

How does MST relate to the 2-approximation for metric TSP?